Universidad Mayor
Facultad de Ciencias, Ingeniería y Tecnología
Escuela de Ingeniería Civil



HITO 2 · DOCUMENTO TÉCNICO DEL BACK END

Agro-Colab

El motor que sostiene la promesa: financia la cosecha, asegura tu abastecimiento.

34	7	139	500
operaciones HTTP	tablas relacionales	pruebas, en 2 motores	compras simultáneas sin sobreventa

Asignatura	Arquitectura de Desarrollo (Móvil y Web) · INGT1003
Equipo	Agro-Colab
Integrantes	Alveal Santos, Joaquin Gonzalez Paredes, Matias Reyes Orellana, Nicolas
Docente	Prof. Carlos Muñoz S.
Ciudad y fecha	Santiago de Chile, 4 de noviembre de 2026

Índice

1 Introducción 1

2 Trazabilidad con el Hito 1 2

3 Arquitectura y decisión monolito o distribuido 4

4 Justificación de las tecnologías 6

5 Diseño de la API REST 7

5.1 Convenciones y mapa de recursos 7

5.2 Contrato completo 7

5.3 Demostración real de crear, leer, actualizar y borrar 9

5.4 Errores: de 200 a su código correcto 10

6 Separación de responsabilidades y capas 12

6.1 Qué archivo corresponde a cada capa 12

6.2 Recorrido completo de una petición 12

6.3 Cómo se hacen cumplir las capas 13

7 Persistencia y manejo de credenciales 16

7.1 Modelo de datos 16

7.2 Reserva atómica de cupos 17

7.3 Credenciales 18

8 Calidad, rendimiento y seguridad 19

9 Limitaciones declaradas 21

10 Conclusión 22

Referencias 23

Anexos 24

SECCIÓN 1

Introducción

> Agro-Colab permite al pequeño agricultor cobrar por adelantado, sin intermediarios. Este documento explica el servicio que hace posible esa promesa y por qué está construido así.

El pequeño agricultor chileno necesita capital antes de la temporada y suele terminar vendiendo su cosecha a intermediarios. En Agro-Colab, el consumidor adopta un árbol frutal o contrata una canasta y paga directamente al productor. El Hito 1 convirtió esa idea en una propuesta y en una arquitectura de tres capas (Alveal et al., 2026).

El objetivo de este documento es describir el motor construido después y defender sus decisiones, incluidas las que todavía tienen un costo. El lector encontrará los cambios respecto del Hito 1, la decisión de arquitectura, el contrato de la API, el recorrido de una petición por las capas, el modelo de datos, las mediciones realizadas y las limitaciones conocidas.

34	7	139/139	91 %
operaciones HTTP en 24 rutas	tablas relacionales persistentes	pruebas aprobadas en SQLite y PostgreSQL	cobertura de líneas del back end

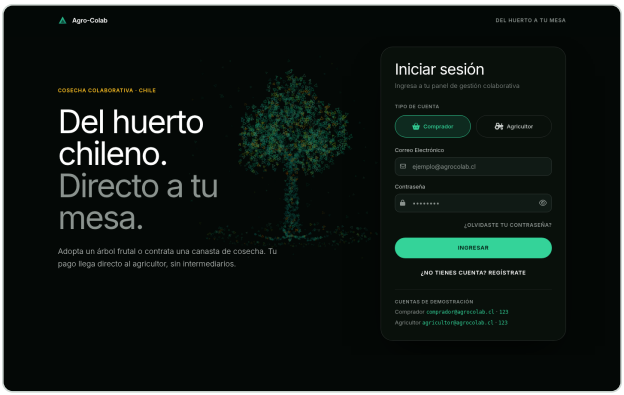


Figura 1. Cliente publicado que consume el servicio descrito en este documento. Captura propia.

Cómo se obtuvieron los datos

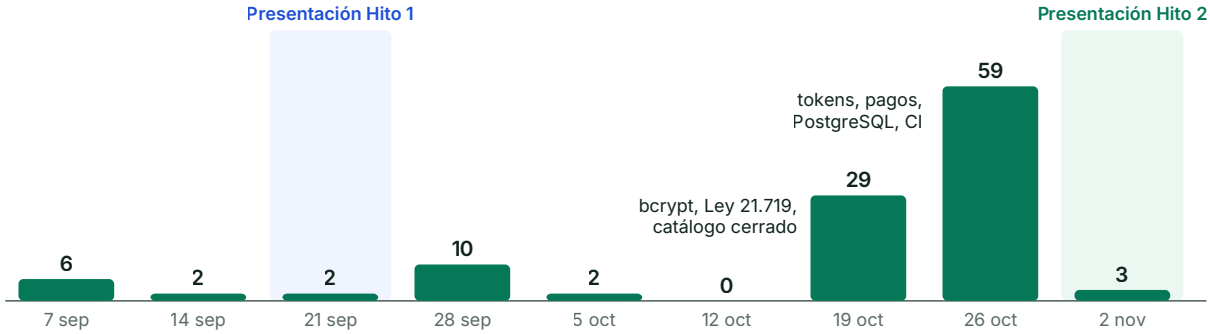
Todas las cifras se midieron el 4 de noviembre de 2026 sobre el código de esta entrega, en un entorno virtual nuevo y con bases temporales. El Anexo D describe el procedimiento.

SECCIÓN 2

Trazabilidad con el Hito 1

> La promesa no cambió; cambió la forma de cumplirla. Cada cambio de esta sección responde a un error real encontrado al construir o a una exigencia de la promesa.

Siguen vigentes el problema, los usuarios, la arquitectura cliente-servidor en tres capas, la API REST en Python con FastAPI, la base relacional y los planes Básico, Intermedio y Premium. La desventaja asumida en el Hito 1 también se cumplió: FastAPI no trae ORM ni panel de administración, así que hubo que integrar SQLAlchemy y programar la seguridad y el panel. La Figura 2 muestra cómo se concentró el trabajo y la Tabla 1 declara qué se movió.



Semana que comienza (lunes). Commits sin merges en la rama principal; total: 113.

Figura 2. Evolución del repositorio por semana hasta el 2 de noviembre, con los temas de las dos semanas de mayor actividad. Datos: historial de Git.

Tabla 1. Cambios respecto del Hito 1, con motivo e impacto.

Qué se movió	Por qué	Impacto
Adoptar exige pagar. `POST /api/suscribir` dio paso a `POST /api/pagos`, pasarela simulada. `30 oct 2026`	La ruta original adoptaba sin pagar y contradecía el financiamiento por adelantado.	Pago y cupo van en una transacción. Sin cupo no hay cobro.
De SQLite a PostgreSQL en producción; SQLite sigue en local. `30 oct 2026`	En Render, SQLite perdía los datos en cada despliegue.	Los datos sobreviven a reinicios. CI prueba ambos motores.
Reserva atómica de cupos. Antes se contaba y luego se insertaba.	Dos compras simultáneas veían el mismo cupo libre: una condición de carrera.	UPDATE condicional e índice único (sección 7.2).
Tokens firmados. El Hito 1 solo decía «verifica que el usuario esté autenticado». `29 oct 2026`	La versión inicial confiaba en el id enviado: cualquiera podía actuar por otra persona.	HMAC-SHA256, expiración de 7 días y autorización por recurso.

Qué se movió	Por qué	Impacto
Catálogo cerrado de 12 especies con fotos verificadas. `21 oct 2026`	La trazabilidad prometida exige mostrar el árbol correcto.	El servidor rechaza especies o fotos ajenas, incluso en peticiones hechas a mano.
Sin vía asíncrona. El Hito 1 justificó FastAPI por su rendimiento asíncrono; ninguna ruta usa `async def`.	El acceso a datos es síncrono. FastAPI ejecuta las rutas `def` en un grupo de hilos (Ramírez, s.f.-a).	El argumento pierde peso. La elección se sostiene por Pydantic y productividad.
Alcance agregado: verificación por correo, recuperación de contraseña, alertas y panel de administración.	Son exigencias de un sistema con cuentas y de la Ley 21.719.	34 operaciones que mantener, compensadas con pruebas.
Contrato HTTP corregido. Los errores de negocio y las creaciones respondían 200. `4 nov 2026`	Una medición mostró que 10 de 17 errores llegaban como éxito. No se había cerrado un contrato de códigos antes de programar.	Cada resultado usa su código (sección 5.4). Se ajustó el cliente y dos rutas cambiaron.
Capas reales. Se separaron `routers/`, `services/` y `repositories/`. `4 nov 2026`	Había 89 líneas con consultas dentro de los routers, y lógica de negocio en el controlador.	Las reglas se comprueban con pruebas automáticas (sección 6.3).
Borrado de cuentas corregido. `4 nov 2026`	En PostgreSQL, borrar un agricultor con despachos de otros clientes respondía 500. SQLite no lo detectaba.	Se borran esos despachos y una prueba de regresión lo protege en ambos motores.

La Tabla 2 revisa, uno por uno, los compromisos técnicos que el Hito 1 dejó escritos. Los seis están cumplidos.

Tabla 2. Compromisos técnicos del Hito 1 y su estado.

Compromiso del Hito 1	Dónde está hoy	Estado
Mostrar el catálogo de árboles	`GET /api/arboles`	`Cumplido`
Verificar que el usuario esté autenticado	`usuario_actual` en `core.py`	`Cumplido`
Controlar los cupos disponibles de cada árbol	`reservar_cupo`, con UPDATE condicional	`Cumplido`
Registrar la suscripción y descontar el cupo en la base	Una transacción junto con el pago	`Cumplido`
Mostrar un panel personal después de adoptar	`GET /api/suscripciones/usuario/{id}`	`Cumplido`
Responder con código de éxito HTTP 200/201	201 al crear y 200 al leer, actualizar o borrar	`Cumplido`

SECCIÓN 3

Arquitectura y decisión monolito o distribuido

> Decisión: monolito modular. Pagar y reservar un cupo deben ocurrir juntos o no ocurrir, y una sola transacción de la base lo garantiza sin coordinar servicios.

La Figura 3 muestra los componentes con sus nombres reales. El cliente es una aplicación de página única; el servidor, un solo proceso de FastAPI organizado en tres capas. Fowler (2015) sostiene que conviene partir con un monolito, porque los límites correctos entre servicios recién se conocen con el uso. La Tabla 3 aplica los cuatro criterios al caso.

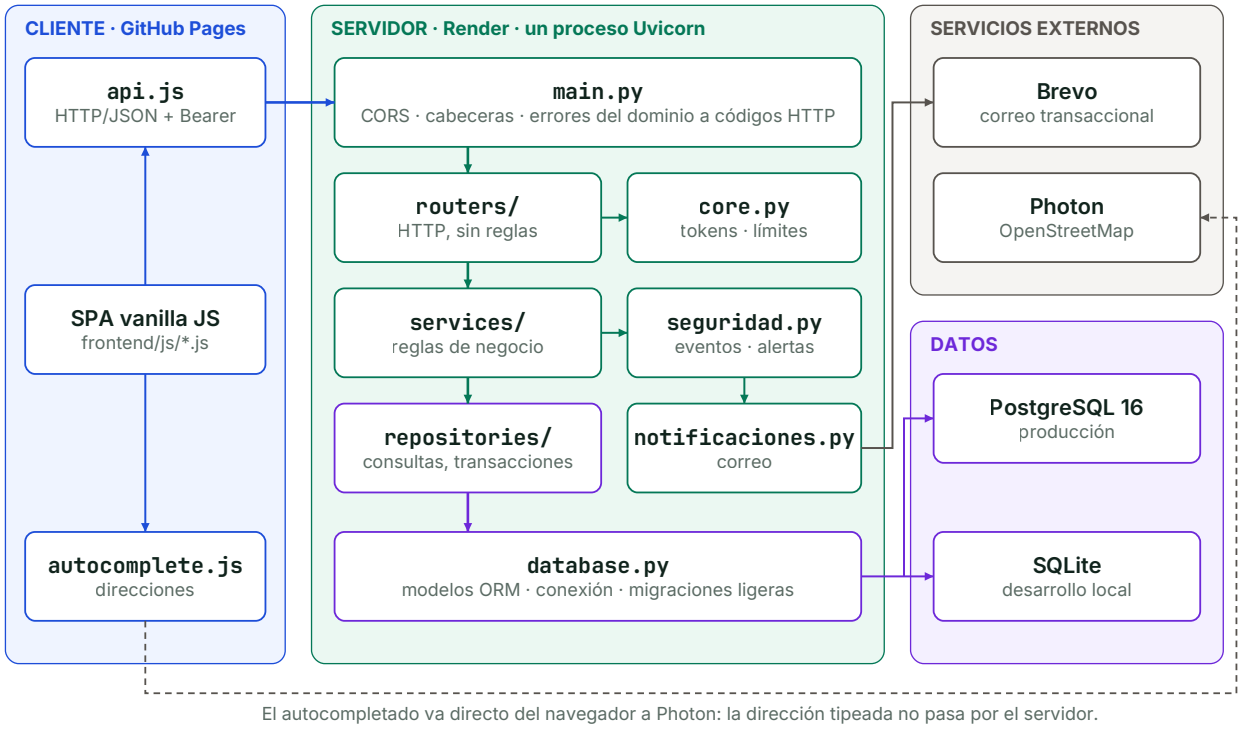


Figura 3. Arquitectura de Agro-Colab. El color identifica la capa y se mantiene en todo el documento. Elaboración propia.

Tabla 3. Monolito frente a servicios, aplicado a Agro-Colab.

Criterio	Situación del proyecto	Con monolito	Con servicios
Carga esperada	Supuesto: decenas de usuarios. Local: entre 104 y 214 lecturas por segundo con 10 clientes (sección 8).	Un proceso alcanza, y los límites de intentos viven en la base, así que se pueden sumar procesos.	Ninguna parte tiene una carga distinta que justifique escalarla aparte.
Tamaño del equipo	3 integrantes y un semestre.	Un repositorio y un conjunto de pruebas.	8 módulos serían demasiados servicios para 3 personas.

Criterio	Situación del proyecto	Con monolito	Con servicios
Costo de operación	Plan gratuito: el servicio se detiene tras 15 minutos sin tráfico (Render, s.f.).	Un servicio web y una base.	Varios servicios detenidos por separado suman esperas en cadena.
Complejidad de despliegue	Pago y cupo son una sola operación de negocio.	Una transacción local (sección 7.2).	Exigiría coordinar una transacción distribuida.

Costo que el equipo asume

Un error grave detiene todo el servicio y no se puede escalar una parte por separado. Además, nada en el proceso impide que un módulo acceda a cualquier tabla: esa disciplina la imponen las pruebas de la sección 6.3, no la infraestructura.

Cuándo se revisaría esta decisión

Correo
Si el envío dentro de la petición retrasa el registro de forma visible: primero, una cola de tareas.

Pagos reales
Una pasarela real trae requisitos de auditoría propios: primer candidato a separarse.

Equipos
Si distintos equipos necesitan publicar de forma independiente.

Carga
Si una medición muestra una parte saturada mientras el resto no lo necesita.

SECCIÓN 4

Justificación de las tecnologías

> Cada pieza se contrastó con una alternativa, usando los criterios del Hito 1: curva de aprendizaje, capacidad frente a peticiones simultáneas, y comunidad y documentación.

La Tabla 4 resume, para cada elección, la alternativa evaluada, el motivo y el costo que el equipo acepta. El motivo que cambió respecto del Hito 1 es el de FastAPI: su valor real resultó ser la validación tipada y la división por dominios, no el rendimiento asíncrono.

Tabla 4. Decisiones de tecnología, alternativas y costos.

Pieza	Alternativa	Por qué se eligió	Costo asumido
Python 3.12 `lenguaje`	Node.js	Sintaxis accesible para el equipo y validación tipada con anotaciones. Node.js habría unificado el lenguaje con el cliente.	Dos lenguajes. Los esquemas no se comparten con el cliente.
FastAPI `framework`	Express, Django	Integra Pydantic, genera `/docs` (Ramírez, s.f.-b) y divide la API por dominio (Ramírez, s.f.-c).	Sin ORM, autenticación ni panel incluidos.
Pydantic `validación`	Validación manual	Tipos y largos se declaran una vez y lo inválido se rechaza con 422 antes de entrar a la ruta (Pydantic, s.f.).	Su mensaje por defecto no servía al usuario: se escribió un manejador (`main.py:82`).
PostgreSQL 16 `producción`	SQLite	Persiste fuera del servidor y documenta su comportamiento concurrente (PostgreSQL Global Development Group, s.f.).	Servicio externo, variable de conexión y pruebas en dos motores.
SQLite `local`	PostgreSQL local	Se levanta sin instalar nada: basta con no definir `DATABASE_URL`.	Diferencias con producción; por eso CI prueba ambos.
SQLAlchemy 2 `ORM`	SQL a mano	Mismo código en ambos motores y consultas parametrizadas (SQLAlchemy, s.f.).	Curva de aprendizaje y migración propia, porque `create_all` no altera tablas.
bcrypt y HMAC-SHA256 `seguridad`	JWT con biblioteca	bcrypt está diseñado para contraseñas (Python Cryptographic Authority, s.f.). HMAC es estándar (Krawczyk et al., 1997) y viene en Python (Python Software Foundation, s.f.).	Código propio de tokens, un punto sensible que exige pruebas.
Render `hospedaje`	Servidor propio	Plan gratuito suficiente para un prototipo, sin administrar un servidor.	La primera petición tras la inactividad demora.

SECCIÓN 5

Diseño de la API REST

> La API cubre crear, leer, actualizar y borrar en dos recursos, persiste en una base real y responde cada resultado con su código HTTP, incluidos todos los errores medidos.

5.1 Convenciones y mapa de recursos

Las rutas usan sustantivos y los verbos de HTTP según su semántica (Fielding, 2000; Fielding et al., 2022). La excepción declarada son las acciones de cuenta, como `/api/login`. La Figura 4 distribuye las 34 operaciones por recurso y verbo.

Formato
JSON. Las escrituras y todos los errores incluyen `status` y `mensaje` en lenguaje de usuario.

Autenticación
`Authorization: Bearer`. El usuario sale del token, nunca del cuerpo ni de la ruta.

Versionado
Sin versión, con un solo consumidor. Un cambio incompatible se publicaría en `/api/v2/`, conviviendo con el anterior hasta migrar al cliente.

Recurso	Módulo	POST crear / acción	GET leer	PUT actualizar	DELETE borrar	Ops.
Árboles CRUD completo	`arboles`	1	2	1	1	5
Pedidos CRUD completo	`pedidos`	1	2	1	1	5
Cuentas y sesión	`usuarios`	6	2	1	1	10
Suscripciones	`suscripciones`	·	2	·	2	4
Administración	`admin`	·	2	1	1	4
Pagos	`pagos`	1	2	·	·	3
Monitoreo	`monitoreo`	1	1	·	·	2
Especies	`especies`	·	1	·	·	1
Total		10	14	4	6	34

Figura 4. Operaciones por recurso y verbo. Árboles y pedidos cumplen el ciclo completo que exige el mínimo funcional.

5.2 Contrato completo

La Tabla 5 es el contrato tal como lo implementa el código, y también está en el repositorio, en `docs/API.md`. Una prueba automatizada falla si ambos dejan de coincidir. Los parámetros son los de ruta, entre llaves; ninguna ruta usa parámetros de consulta, y

`/api/seguridad/resumen` exige además la cabecera `X-Monitoreo-Token`. La **Tabla 6** resume las reglas de validación de los cuerpos.

Tabla 5. Contrato de la API. Las rutas con cuerpo o parámetro numérico responden 422 ante datos inválidos.

Verbo	Ruta	Acceso	Cuerpo	Éxito	Errores
Especies					
GET	`/api/especies`	Público	—	200	—
Árboles					
GET	`/api/arboles`	Público	—	200	—
GET	`/api/arboles/{arbol_id}`	Público	—	200	404 422
POST	`/api/arboles`	Agricultor	ArbolRequest	201	401 403 422
PUT	`/api/arboles/{arbol_id}`	Su productor	ArbolRequest	200	401 403 404 409 422
DELETE	`/api/arboles/{arbol_id}`	Su productor	—	200	401 403 404 422
Suscripciones					
GET	`/api/suscripciones/usuario/{usuario_id}`	El mismo usuario	—	200	401 403 422
GET	`/api/mis-arboles/{productor_id}/suscriptores`	El mismo productor	—	200	401 403 422
DELETE	`/api/arboles/{arbol_id}/suscripcion`	Comprador suscrito	—	200	401 404 422
DELETE	`/api/suscripciones/{suscripcion_id}`	Productor del árbol	—	200	401 403 404 422
Pedidos					
GET	`/api/pedidos`	Sesión	—	200	401
GET	`/api/pedidos/{pedido_id}`	Su cliente	—	200	401 404 422
POST	`/api/pedidos`	Comprador	PedidoRequest	201	401 403 409 422
PUT	`/api/pedidos/{pedido_id}`	Su cliente	LogisticaRequest	200	401 404 422
DELETE	`/api/pedidos/{pedido_id}`	Su cliente	—	200	401 404 409 422
Pagos (pasarela simulada)					
GET	`/api/pagos`	Sesión	—	200	401
GET	`/api/pagos/{pago_id}`	Su titular	—	200	401 404 422
POST	`/api/pagos`	Comprador	PagoRequest	201	401 402 403 404 409 422 429
Cuentas y sesión					
POST	`/api/register`	Público	RegistroRequest	202	422 429 503
POST	`/api/verificar-cuenta`	Público	{email, codigo}	200	400 422 429
POST	`/api/reenviar-codigo`	Público	{email}	200	422 429
POST	`/api/login`	Público	LoginRequest	200	401 422 429
POST	`/api/recuperar-contrasena`	Público	{email}	200	422 429 503

Verbo	Ruta	Acceso	Cuerpo	Éxito	Errores
POST	/api/restablecer-contrasena	Público	{email, codigo, password}	200	400 422 429
GET	/api/estado	Público	—	200	—
GET	/api/usuario/{usuario_id}	Él mismo o su productor	—	200	401 403 404 422
PUT	/api/perfil	Sesión	PerfilRequest	200	401 403 422
DELETE	/api/perfil	Sesión	—	200	401 403
Administración					
GET	/api/admin/usuarios	Administrador	—	200	401 403
GET	/api/admin/usuarios/{usuario_id}	Administrador	—	200	401 403 404 422
PUT	/api/admin/usuarios/{usuario_id}	Administrador	AdminUsuarioRequest	200	401 403 404 409 422
DELETE	/api/admin/usuarios/{usuario_id}	Administrador	—	200	401 403 404 422
Monitoreo de seguridad					
POST	/api/seguridad/csp	Público (navegador)	ReporteCSP	204	422 429
GET	/api/seguridad/resumen	Token de monitoreo	—	200	401 404 429

Tabla 6. Reglas de validación de los esquemas principales.

Esquema	Campos y reglas
ArbolRequest	especie del catálogo; plan Básico, Intermedio o Premium; cupos de 1 a 50; precio de 0 a 10.000.000; cosecha de 1 a 100.000 kg; imagen de la especie.
PedidoRequest	destino particular o comercial; frecuencia semanal, quincenal o mensual; RUT con dígito verificador; pago aprobado. El monto enviado se ignora.
PagoRequest	tipo arbol o plan; número con Luhn; vencimiento MM/AA; CVV de 3 o 4 dígitos.
RegistroRequest	nombre de 2 a 80; correo con formato; contraseña de 8 a 128; rol comprador o agricultor.

```
Cuerpo de ejemplo · POST /api/arboles · responde 201

{"especie": "palto_hass", "identificador": "#7", "ubicacion": "Quillota",
 "plan": "Básico", "tipo_suscripcion": "Compartido", "cupos_totales": 2,
 "precio_mensual_clp": 9000, "estimacion_cosecha_kg": 30,
 "imagen": "<url de una foto de la especie>"}
```

5.3 Demostración real de crear, leer, actualizar y borrar

La Figura 5 reproduce, con el cuerpo abreviado, la secuencia que se mostrará en vivo contra PostgreSQL. Después de borrar, leer el mismo pedido responde 404: el dato estaba guardado y dejó de estarlo. La colección de peticiones está en `docs/AgroColab.postman_collection.json`, y la Figura 6 muestra la documentación interactiva que genera el framework.

```
$ curl -i -X POST $API/pedidos -H "Authorization: Bearer $TOKEN" -d '{..., "pago_id": 1}'
HTTP/1.1 201 Created
{"status":"success","mensaje":"Pedido registrado. Código de despacho: AGRO-0001",
 "pedido":{"id":1,"monto_clp":19900,"comuna":"Ñuñoa","frecuencia_entrega":"mensual", ...}}

$ curl -i $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
HTTP/1.1 200 OK
{"id":1,"plan_nombre":"Canasta Huerto Esencial","comuna":"Ñuñoa",
 "codigo_seguimiento":"AGRO-0001", ...}

$ curl -i -X PUT $API/pedidos/1 -H "Authorization: Bearer $TOKEN" -d '{..., "comuna": "Macul"}'
HTTP/1.1 200 OK
{"status":"success","mensaje":"Logística actualizada para AGRO-0001.",
 "pedido":{"id":1,"comuna":"Macul", ...}}

$ curl -i -X DELETE $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
HTTP/1.1 200 OK
{"status":"success","mensaje":"Plan Canasta Huerto Esencial cancelado."}

$ curl -i $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
HTTP/1.1 404 Not Found
{"status":"error","mensaje":"Pedido no encontrado."}
```

Figura 5. Ciclo completo sobre el recurso pedidos, capturado del servicio en ejecución con datos ficticios. El Anexo B trae los comandos completos.

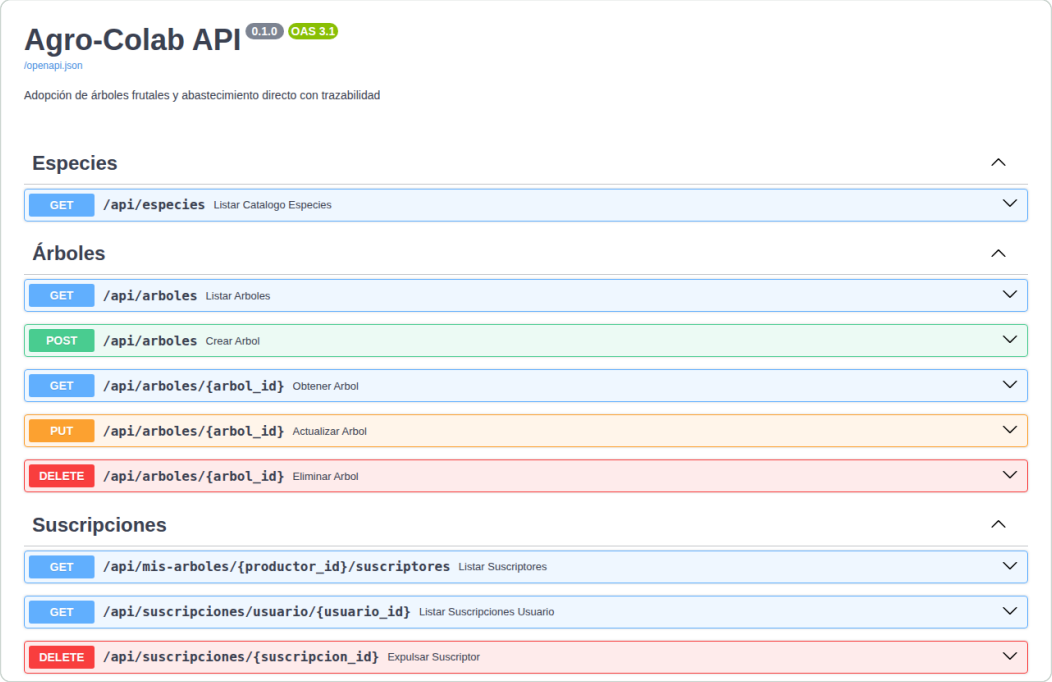


Figura 6. Documentación interactiva en /docs (vista parcial), capturada del servicio en ejecución.

5.4 Errores: de 200 a su código correcto

Medir la API mostró un defecto que la interfaz ocultaba: el cliente leía `status`, y por eso 10 de 17 errores respondían 200 sin que nadie lo notara. Eso contradice la semántica de HTTP (Fielding et al., 2022) y engaña a cualquier otro consumidor. Se corrigió, y las

mismas 29 peticiones dan hoy el resultado de la Figura 7. La Tabla 7 detalla los 17 errores.

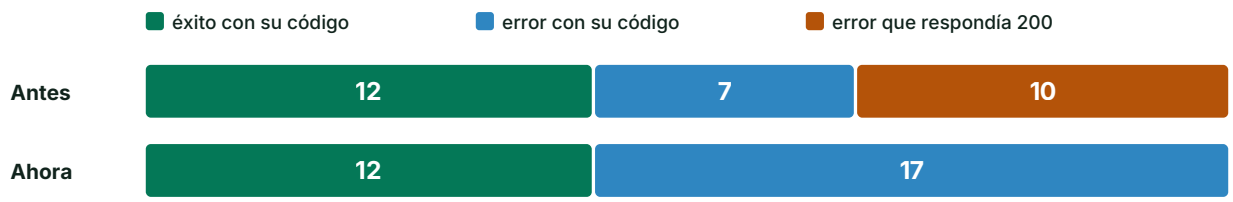


Figura 7. Las mismas 29 peticiones reales, antes y después de la corrección. Los éxitos que crean un recurso pasaron de 200 a 201.

Tabla 7. Escenarios de error: código antes y código medido ahora. `200*` significa 200 con el error en el cuerpo.

Escenario	Petición	Antes	Ahora
Sin token	`POST /api/arboles`	401	401
Token falso	`POST /api/arboles`	401	401
Datos de otro usuario	`GET /api/suscripciones/usuario/2`	403	403
Panel de administración sin rol	`GET /api/admin/usuarios`	403	403
Árbol con 0 cupos	`POST /api/arboles`	422	422
Ruta inexistente	`GET /api/nada`	404	404
Verbo no admitido	`PATCH /api/arboles`	405	405
Contraseña incorrecta	`POST /api/login`	200*	401
Comprador publica un árbol	`POST /api/arboles`	200*	403
Especie inexistente	`POST /api/arboles`	200*	422
Editar árbol inexistente	`PUT /api/arboles/9999`	200*	404
Adoptar dos veces	`POST /api/pagos`	200*	409
Tarjeta rechazada	`POST /api/pagos`	200*	402
Pedido sin pago	`POST /api/pedidos`	200*	409
Editar pedido inexistente	`PUT /api/pedidos/9999`	200*	404
Cancelar pedido ya cancelado	`DELETE /api/pedidos/1`	200*	404
Cuenta demo borra su árbol	`DELETE /api/arboles/4`	200*	403

RFC 9110 reserva el código 402 para uso futuro (Fielding et al., 2022), aunque es la convención habitual de las pasarelas. El equipo lo adoptó y lo declara en el contrato: el cuerpo trae `status: rechazado` y el comprobante.

La sección 6.3 muestra cómo la capa de negocio y el manejador de errores evitan que este defecto vuelva a aparecer.

SECCIÓN 6

Separación de responsabilidades y capas

> El ruteo, el negocio y el acceso a datos viven en carpetas separadas, y una prueba automática falla si alguien vuelve a mezclarlos.

6.1 Qué archivo corresponde a cada capa

La Figura 8 asigna cada archivo a su capa. La regla se definió antes de mover el código: el controlador no lleva lógica de negocio, y las consultas viven solo en la capa de datos.

agro-colab/		
├─ frontend/	Presentación	SPA, una sola página
│ └─ js/api.js	Presentación	único punto que llama al servidor
├─ backend/		
│ ├─ main.py	Aplicación	CORS, cabeceras, errores a códigos HTTP
│ ├─ routers/	HTTP	8 módulos: rutas, sin reglas ni consultas
│ ├─ schemas.py	HTTP	contratos de entrada con Pydantic
│ ├─ services/	Negocio	reglas de cuentas, árboles, pagos y pedidos
│ ├─ errores.py	Negocio	errores del dominio, sin HTTP
│ ├─ catalogo.py	Negocio	12 especies y 36 fotos verificadas
│ ├─ repositories/	Datos	únicas consultas y transacciones
│ ├─ database.py	Datos	conexión, modelos, migraciones
│ ├─ core.py	Transversal	tokens, autenticación, límites
│ ├─ seguridad.py	Transversal	eventos y reglas de alerta
│ ├─ notificaciones.py	Integración	correo con Brevo
│ └─ tests/	Pruebas	139 pruebas con pytest
├─ db/	Datos	diagrama del modelo y script SQL
├─ docs/	Pruebas	este informe, API.md y colección de Postman
└─ .github/workflows/pruebas.yml	CI	tres jobs en cada PR

Figura 8. Estructura del repositorio con la capa de cada archivo.

6.2 Recorrido completo de una petición

El caso de uso es el mismo que recorrió el Hito 1: una persona adopta un árbol. La Figura 9 lo sigue por el código real, desde la ruta hasta la base. El orden importa: FastAPI resuelve primero la dependencia `usuario_actual`, así que una petición sin token e inválida recibe 401 y no 422, lo que también comprueba una prueba automatizada. Si un servicio lanza un error, `main.py` lo traduce a su código.

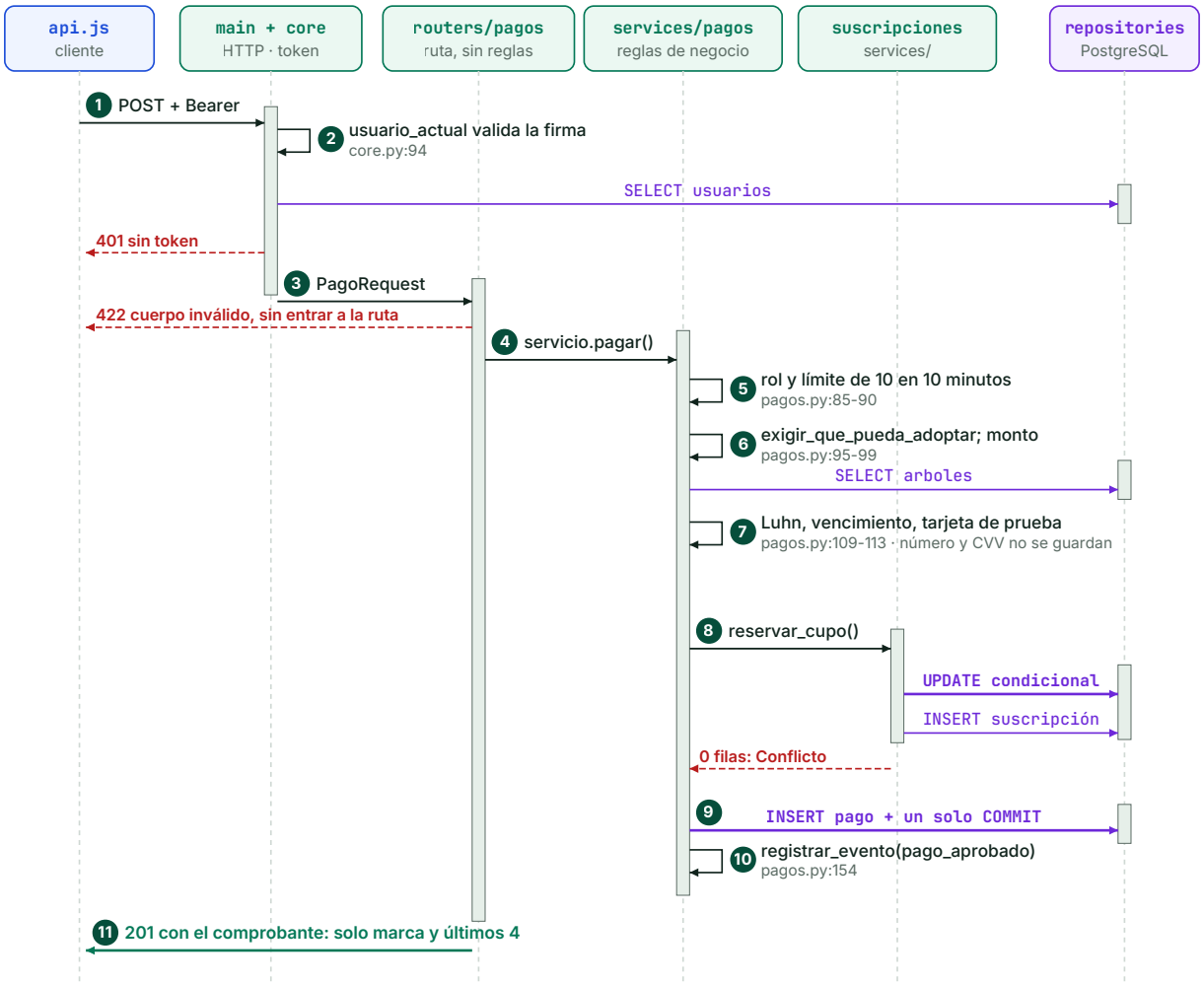


Figura 9. Secuencia de `POST /api/pagos` con referencias al código. Las líneas violetas son SQL; las rojas, salidas de error.

6.3 Cómo se hacen cumplir las capas

Una primera medición encontró 89 líneas con operaciones de base de datos dentro de los routers, y lógica de negocio en el controlador. Se separó el código en tres capas, como muestra la Figura 10. Las pruebas existentes protegieron el cambio porque verifican la API desde afuera: 47 de 49 siguieron pasando sin tocarlas, y las otras dos correspondían a las rutas que se renombraron a propósito.

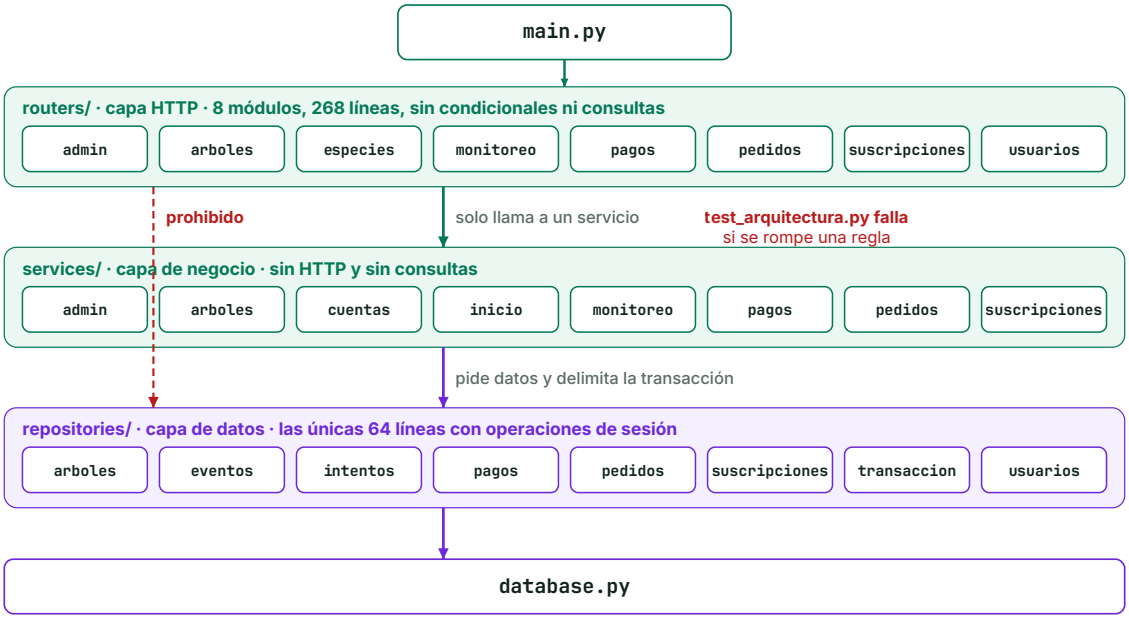


Figura 10. Capas del servidor y dependencias permitidas. Los módulos transversales se describen en la Figura 8.

La Figura 11 cuantifica el resultado y la Figura 12 lo muestra para un caso. Para que no se deshaga, `test_arquitectura.py` analiza el árbol sintáctico del código en cada integración y comprueba cinco reglas, según la Tabla 8.



Figura 11. Líneas con operaciones directas de sesión (`db.query`, `add`, `delete`, `commit`, `execute`, `flush`, `refresh`, `rollback`).

Tabla 8. Reglas de capas que se comprueban automáticamente.

Regla	Qué falla si se rompe
Las consultas viven solo en la capa de datos	Una llamada como `db.query` en routers, servicios, `core.py` o `seguridad.py`
Los routers no saltan la capa de negocio	Un router que importa `repositories`
Los routers no contienen reglas	Un `if`, ciclo o `try` dentro de una ruta
El negocio no conoce HTTP	Un servicio que importa FastAPI o un router
Los datos no dependen de capas superiores	Un repositorio que importa servicios, routers o `core.py`

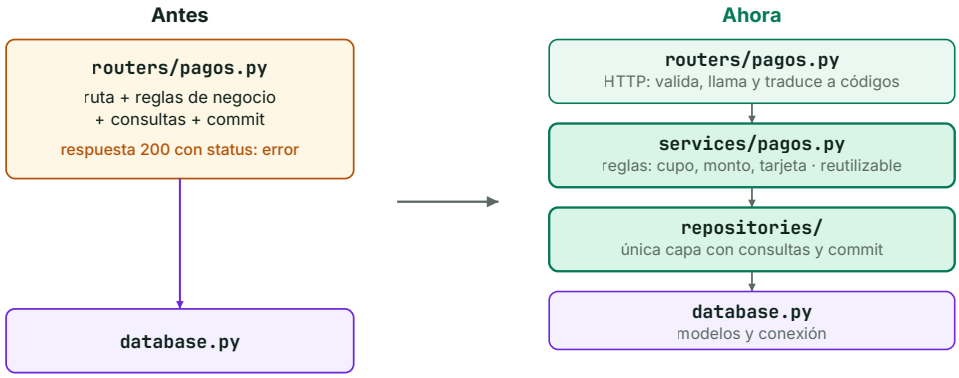


Figura 12. El módulo de pagos antes y después de la separación. En verde intenso, las capas nuevas. Elaboración propia.

La capa de negocio lanza errores del dominio sin saber nada de HTTP, y un solo manejador los traduce (Ramírez, s.f.-d), como muestra el código siguiente. Así, ningún servicio puede responder un error con código de éxito.

```
backend/main.py (extracto)

CODIGOS_HTTP = {
    errores.CodigoIncorrecto: 400, errores.NoAutenticado: 401, errores.PagoRechazado: 402,
    errores.Prohibido: 403, errores.NoEncontrado: 404, errores.Conflicto: 409,
    errores.DatoInvalido: 422, errores.DemasiadosIntentos: 429, errores.ServicioNoDisponible: 503,
}

@app.exception_handler(errores.ErrorDeNegocio)
async def error_de_negocio(request, exc):
    return respuesta_de_error(CODIGOS_HTTP[type(exc)], exc.mensaje, exc.estado, **exc.extra)
```

Lo que estas capas no resuelven

Los servicios modifican directamente los objetos del ORM y los routers importan `Usuario` y `Session` como anotaciones de tipo, así que los modelos no están aislados por completo. Se aceptó ese costo: agregar objetos de dominio separados no se justifica en un proyecto de este tamaño.

SECCIÓN 7

Persistencia y manejo de credenciales

> Los datos persisten en PostgreSQL, las credenciales viven fuera del código y la garantía más delicada, no vender dos veces el último cupo, se comprobó con 500 compras simultáneas.

7.1 Modelo de datos

La Figura 13 muestra las 7 tablas con claves, relaciones y tipos. Una suscripción es la relación N a N entre usuarios y árboles. `pagos.arbol_id` y `pagos.pedido_id` no son claves foráneas a propósito: el comprobante debe sobrevivir si el agricultor elimina el árbol.

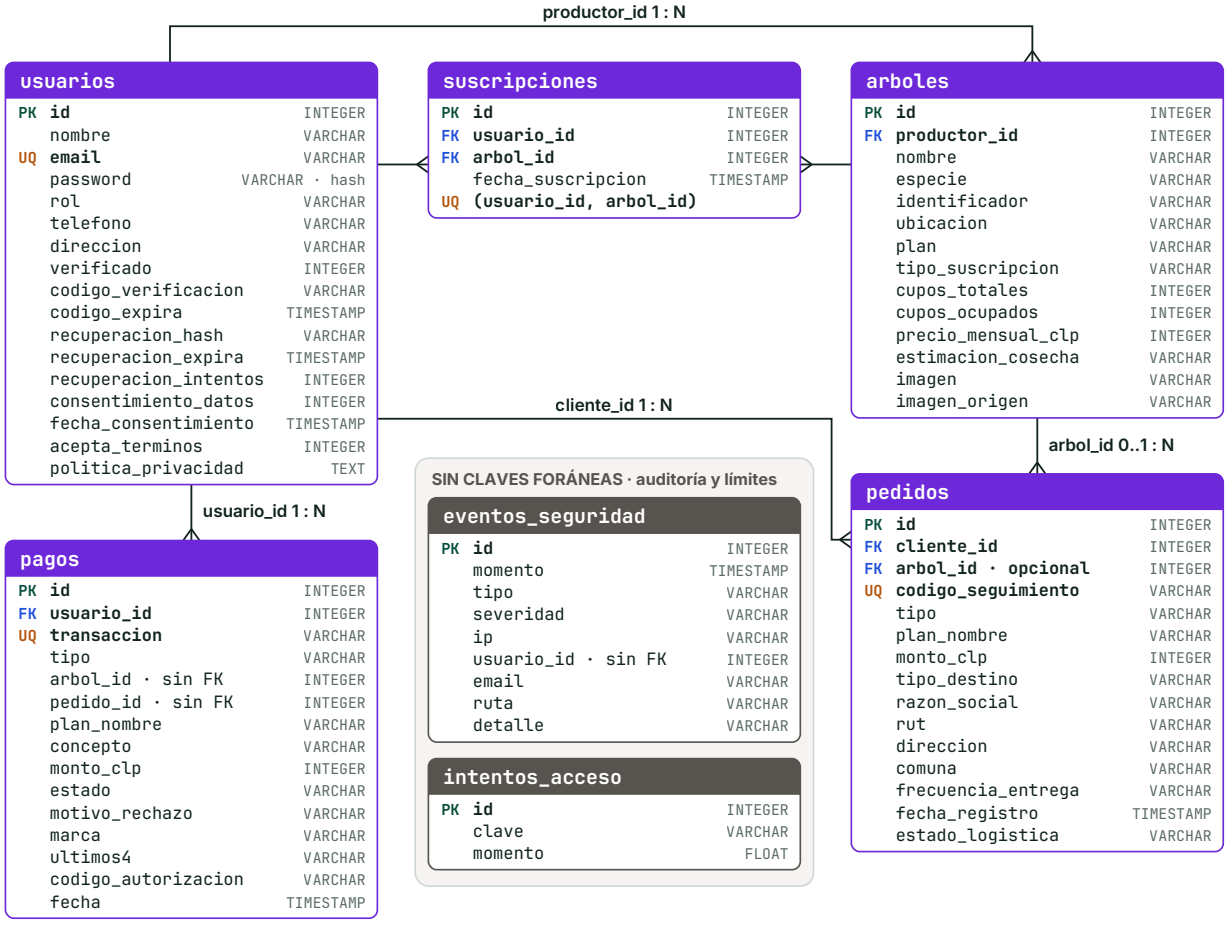


Figura 13. Modelo de datos completo, derivado de `backend/database.py`. PK: clave primaria; FK: clave foránea; UQ: valor único.

La base se crea desde cero al arrancar: `create_all()` crea las tablas (`main.py:137`), `asegurar_columnas()` agrega columnas nuevas a bases existentes y `datos_iniciales()` carga las cuentas de demostración y tres árboles. La carpeta `db/` contiene el diagrama y

un script que crea la base desde cero, reproducido en el Anexo C; una prueba automatizada comprueba que coincida con los modelos, y el script se ejecutó en PostgreSQL 16, donde creó las 7 tablas.

7.2 Reserva atómica de cupos

La promesa depende de que dos compras no se queden con el mismo cupo. Por eso la reserva no consulta y luego escribe: lo hace todo en una sentencia, en la capa de datos.

```
backend/repositories/arboles.py

def reservar_cupo(db: Session, arbol_id: int) → bool:
    filas = db.execute(
        update(Arbol)
        .where(Arbol.id == arbol_id, Arbol.cupos_ocupados < Arbol.cupos_totales)
        .values(cupos_ocupados=Arbol.cupos_ocupados + 1)
    ).rowcount
    return bool(filas)
```

En PostgreSQL, si otra transacción modificó la fila, la segunda espera y luego reevalúa el `WHERE` sobre la versión actualizada (PostgreSQL Global Development Group, s.f.). La Figura 14 lo ilustra. El índice único `uq_suscripcion_usuario_arbol` impide además que una persona ocupe dos cupos del mismo árbol.

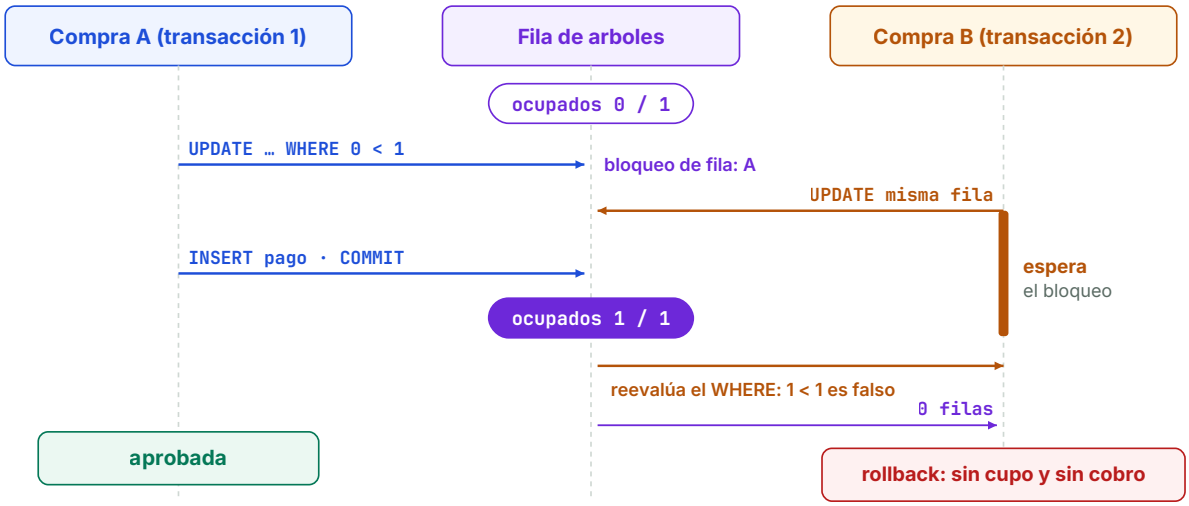


Figura 14. Dos compras simultáneas del último cupo. Elaboración propia a partir de la documentación de PostgreSQL.

Prueba realizada contra PostgreSQL 16.2, con 4 procesos Uvicorn

Último cupo: 20 rondas de 25 compradores distintos atacando a la vez un árbol de 1 cupo.
Mismo comprador: 10 rondas de 10 pagos simultáneos sobre un árbol de 5 cupos. En cada ronda se verificó el estado final en la base.

500	20 de 20	0	10 de 10
compras simultáneas: 20 con 201 y 480 con 409	rondas con exactamente 1 aprobada	respuestas 5xx y 0 inconsistencias	rondas con 1 solo cupo por comprador

7.3 Credenciales

Las credenciales viven en variables de entorno, como recomienda la metodología de doce factores (Wiggins, s.f.), y el código no contiene claves reales. Se revisó todo el historial buscando claves de Brevo, cadenas de conexión con contraseña y archivos `.env`, sin encontrar credenciales de producción; lo que aparece son valores ficticios de las pruebas. El archivo `.env.example` declara todas las variables con valores de ejemplo, `.gitignore` excluye `.env`, y una prueba falla si aparece una variable sin declarar. La Tabla 9 las lista.

Tabla 9. Variables de entorno del servidor.

Variable	Obligatoria	Uso
`DATABASE_URL`	En producción	Conexión a PostgreSQL. Sin ella se usa SQLite local.
`AGROCOLAB_DB_URL`	No	Tiene prioridad sobre la anterior. La usan las pruebas para una base temporal.
`AGROCOLAB_SECRET`	En producción	Firma de tokens. Sin ella se genera una al arrancar y las sesiones se cierran en cada reinicio.
`AGROCOLAB_MOD_MODE`	No	`desarrollo` muestra el código de verificación sin enviar correo.
`BREVO_API_KEY` `CORREO_REMITENTE`	Para registrar	Correos transaccionales.
`ADMIN_EMAIL` `ADMIN_PASSWORD` `ADMIN_NOMBRE`	Para el panel	Cuenta de administrador; la contraseña exige 12 caracteres o más.
`MONITOREO_TOKEN`	Para el panel	24 caracteres o más. Sin él, el monitoreo queda desactivado.
`ALERTA_CORREO` `ALERTA_WEBHOOK_URL`	No	Destino de las alertas de seguridad.

Las contraseñas se guardan con bcrypt. Las cuentas de demostración usan a propósito una contraseña pública, y por eso el servidor les prohíbe modificar o eliminar sus datos principales.

SECCIÓN 8

Calidad, rendimiento y seguridad

> Las pruebas protegen el contrato y las capas en dos motores, y una medición local reemplaza el supuesto de carga por un dato, con sus límites declarados.

Las 139 pruebas se ejecutan con pytest contra la API, con una base temporal y sin enviar correos. Todas se aprobaron en SQLite y en PostgreSQL 16, en cerca de 30 segundos cada vez. Una de ellas ejecuta completa la colección de peticiones de la demostración. GitHub Actions (GitHub, s.f.) repite ese proceso en cada solicitud de incorporación, según la Figura 15. La cobertura de líneas del back end es de 91 %, y la Figura 16 muestra dónde falta: `notificaciones.py` queda baja porque las pruebas nunca envían correos reales.



Figura 15. Integración continua definida en `.github/workflows/pruebas.yml`.

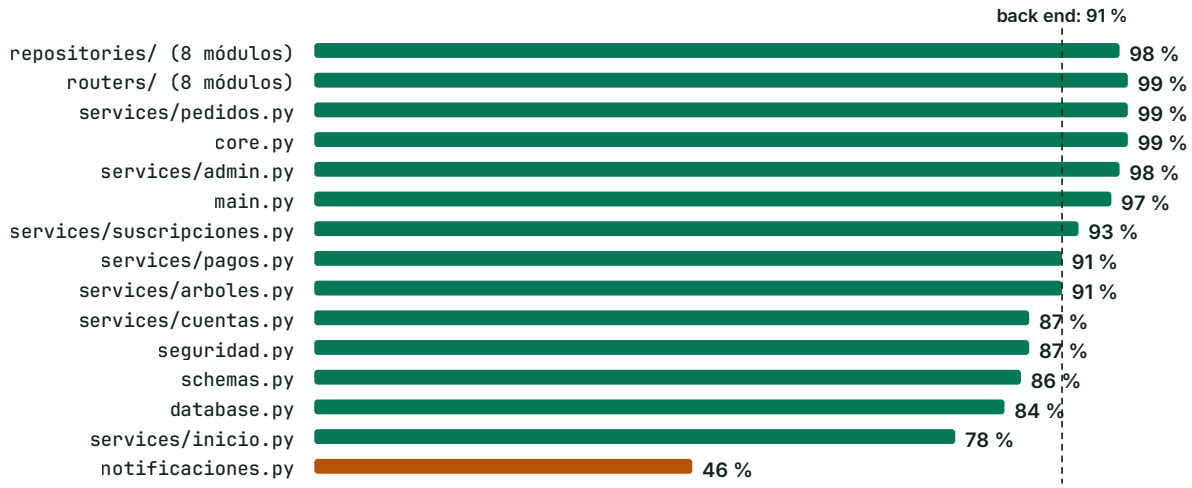


Figura 16. Cobertura de líneas por módulo, medida con pytest-cov. En ámbar, los módulos bajo 65 %. Las dos primeras filas agrupan varios módulos.

Rendimiento

La Figura 17 resume 2.090 peticiones contra el servicio local con 4 procesos y PostgreSQL 16.2. Con 10 clientes, las lecturas sostienen entre 104 y 214 peticiones por segundo, y los pagos, 131. Con 50 clientes aparece la cola: el p95 supera los 450 ms.

Para decenas de usuarios el margen es amplio, lo que respalda el monolito de la sección 3.

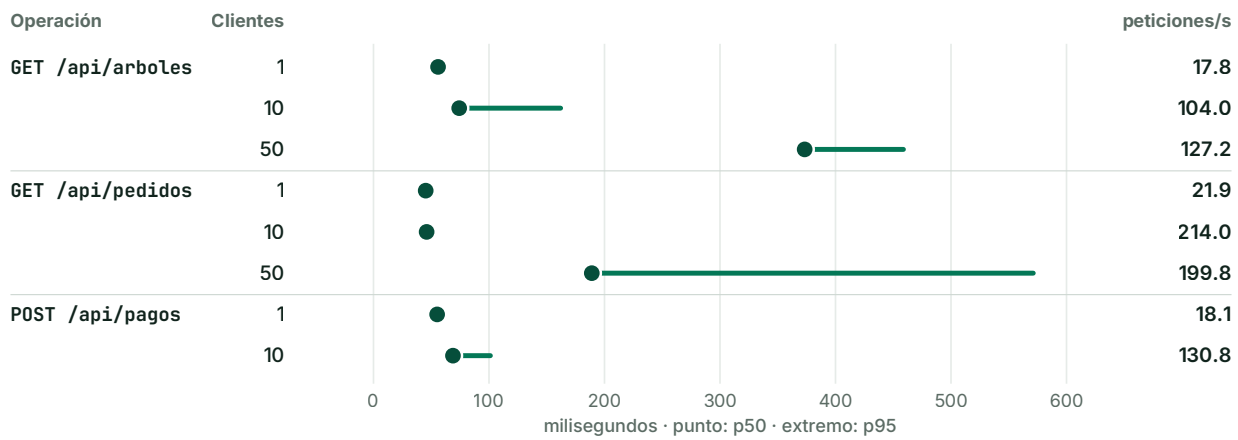


Figura 17. Latencia y rendimiento por operación y número de clientes concurrentes.

> **Límite de esta medición**
> El cliente y el servidor corrieron en el mismo equipo (Anexo D). Las cifras comparan operaciones y muestran la tendencia, pero no predicen el plan gratuito de Render, que tiene otros recursos y otra red.

Controles de seguridad

La Tabla 10 relaciona cada amenaza con el control implementado y su evidencia. El límite por cuenta también se observó en la medición: de 60 intentos de inicio de sesión desde direcciones distintas, 10 pasaron y 50 recibieron 429.

Tabla 10. Amenazas, controles y evidencia.

Amenaza	Control	Evidencia
Robo de contraseñas	bcrypt; cambiarla exige la actual y cierra las demás sesiones	`test_cuentas.py`
Suplantación	Token HMAC-SHA256 de 7 días, ligado a la contraseña	`test_cuentas.py` y Tabla 7
Acceso a datos ajenos	Autorización por recurso; el usuario sale del token	`test_seguridad.py` y medición 403
Fuerza bruta	Límite por IP y por cuenta, persistido en la base	`test_seguridad.py` y medición 429
Manipulación de precios	El monto lo fija el servidor	`test_pagos.py`
XSS	Rechazo de < y >, escape en el cliente y CSP sin `unsafe-inline`	`test_seguridad.py`
Exposición de tarjetas	Solo se guardan marca y últimos 4 dígitos	`test_pagos.py`
Ataques sin detectar	Eventos en base y logs JSON, con alertas por reglas	Panel de monitoreo

SECCIÓN 9

Limitaciones declaradas

> El equipo prefiere declarar lo que falta a que se descubra en la revisión. Cada limitación tiene su consecuencia y su corrección.

La Tabla 11 lista las limitaciones vigentes, ordenadas por su efecto sobre quien consume el servicio.

Tabla 11. Limitaciones conocidas del servicio.

Limitación	Consecuencia	Cómo se corregiría
Columnas sin NOT NULL y `verificado` como entero	La base acepta valores vacíos que solo frena la aplicación.	Declarar restricciones con una migración versionada.
Migración propia al arrancar	Solo agrega columnas: no renombra ni cambia tipos.	Adoptar migraciones versionadas. Es lo que frena el punto anterior, porque hay datos en producción.
Rutas de lectura poco uniformes, como `/api/mis-arboles/{id}/suscriptores`	Contrato menos regular, aunque correcto.	Unificarlas bajo un solo recurso en una futura versión.
Modelos sin aislar por completo	Los servicios modifican objetos del ORM directamente (sección 6.3).	Objetos de dominio, si el proyecto creciera.
Medición de carga solo local	No predice el plan gratuito de Render.	Medir en el entorno publicado antes de revisar la arquitectura.
Espera inicial en Render	Tras 15 minutos sin tráfico, la primera petición demora.	Plan pagado, si sale de la etapa académica.
Correo dentro de la petición	Un envío lento retrasa el registro hasta 10 segundos.	Cola de tareas (sección 3).
Pasarela simulada	No mueve dinero y solo acepta tarjetas de prueba.	Pasarela real, lo que reabre la decisión de arquitectura.

El sistema trata datos personales y la Ley 21.719 entra en plena vigencia el 1 de diciembre de 2026 (Ley N.° 21.719, 2024). El análisis completo corresponde a la documentación del Hito 3. Hoy el servicio ya aplica hash de contraseñas, control de acceso por recurso, registro de accesos con retención de 30 días y eliminación de la cuenta a petición del titular, y el correo ya no viaja en la ruta de eliminación.

SECCIÓN 10

Conclusión

Qué se logró

El motor prometido en el Hito 1 funciona: recursos con sus cuatro operaciones, persistencia real en PostgreSQL, credenciales fuera del código, capas separadas y un servicio que se levanta siguiendo las instrucciones. Cada resultado responde con su código HTTP. La promesa técnica más delicada quedó garantizada por la base de datos, no por una revisión previa, y se comprobó con 500 compras simultáneas.

Qué quedó pendiente

Las restricciones de la base requieren migraciones versionadas, y la carga debe medirse en el entorno publicado. El Hito 3 parte de un contrato estable: el cliente ya consume estos códigos y una prueba impide que el contrato y su documentación se separen.

Qué se aprendió

Los errores más instructivos no se veían en pantalla: un cupo vendido dos veces, datos perdidos en cada despliegue, un id de usuario aceptado sin verificar, y un borrado que fallaba solo en PostgreSQL. También quedó claro el costo de programar sin un contrato cerrado: el cliente aprendió a leer `status` y los códigos HTTP dejaron de importar hasta que se midieron. Las pruebas que miran la API desde afuera hicieron seguro reorganizar todo el servidor. Y una decisión bien tomada puede sostenerse por motivos distintos a los que se creía: FastAPI no se justificó por el rendimiento asíncrono, sino por su validación.

Referencias

Alveal, J., Gonzalez, M., y Reyes, N. (2026). *Agro-Colab: propuesta de valor y arquitectura (Hito 1)* [Informe de asignatura]. Universidad Mayor. https://github.com/arrobaaas/agro-colab

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., y Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

Fowler, M. (2015, 3 de junio). *MonolithFirst*. martinfowler.com. https://martinfowler.com/bliki/MonolithFirst.html

GitHub. (s.f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

Krawczyk, H., Bellare, M., y Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2104

Ley N.° 21.719. Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. (2024). Biblioteca del Congreso Nacional de Chile. https://www.bcn.cl/leychile/navegar?idNorma=1209272

PostgreSQL Global Development Group. (s.f.). Transaction isolation. En *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/transaction-iso.html

Pydantic. (s.f.). *Pydantic documentation*. https://docs.pydantic.dev/latest/

Python Cryptographic Authority. (s.f.). *bcrypt*. Python Package Index. https://pypi.org/project/bcrypt/

Python Software Foundation. (s.f.). hmac: Keyed-hashing for message authentication. En *Python documentation*. https://docs.python.org/3/library/hmac.html

Ramírez, S. (s.f.-a). *Concurrency and async / await*. FastAPI. https://fastapi.tiangolo.com/async/

Ramírez, S. (s.f.-b). *FastAPI*. https://fastapi.tiangolo.com/

Ramírez, S. (s.f.-c). *Bigger applications: Multiple files*. FastAPI. https://fastapi.tiangolo.com/tutorial/bigger-applications/

Ramírez, S. (s.f.-d). *Handling errors*. FastAPI. https://fastapi.tiangolo.com/tutorial/handling-errors/

Render. (s.f.). *Deploy for free*. https://render.com/docs/free

SQLAlchemy. (s.f.). *SQLAlchemy 2.1 documentation*. https://docs.sqlalchemy.org/en/21/

Wiggins, A. (s.f.). *Config*. The Twelve-Factor App. https://12factor.net/config

Anexos

Anexo A. Uso de inteligencia artificial

Tabla 12. Herramientas de inteligencia artificial usadas.

Herramienta	Período	En qué parte	Cómo se verificó
Gemini (Google)	Sept. 2026	Argumentos comerciales del informe del Hito 1.	Lectura de las fuentes citadas en ese informe.
ChatGPT (OpenAI)	Sept. 2026	Comparación de Node.js y Python, y prototipo inicial.	Revisión del código antes de incorporarlo.
Claude Code (Anthropic)	Oct. 2026	Revisión de código, autenticación y autorización, condición de carrera y documentación.	Peticiones a la API: sin token, con token falso, a recursos ajenos y compras simultáneas.
Claude Code (Anthropic)	Nov. 2026	Análisis contra la rúbrica; corrección de códigos HTTP y separación en capas; pruebas nuevas; carpeta db/ y colección; scripts de medición, redacción y diagramas de este documento.	139 pruebas en dos motores, reglas de capas automatizadas, 29 peticiones reales, concurrencia y carga, instalación limpia, cliente en un navegador real y lectura de las fuentes clave.

La responsabilidad sobre el contenido y el código es del equipo, y cualquier integrante debe poder explicarlos.

Anexo B. Guion de la demostración en vivo

La Figura 18 propone el reparto de los 7 minutos, en el orden que pide el enunciado. Las peticiones cubren crear, leer, actualizar y borrar sobre `pedidos`, porque la cuenta demo de agricultor no puede borrar árboles. Se probaron en este orden sobre una base nueva, y también están en la colección de Postman del repositorio.



Figura 18. Reparto propuesto de los 7 minutos de presentación.

```
# Backend en otra terminal: AGROCOLAB_MOD_MODE=desarrollo uvicorn main:app --port 8000
export API=http://127.0.0.1:8000/api
# 0. Iniciar sesión. Copiar user.token en TOKEN
curl -i -X POST $API/login -H 'Content-Type: application/json' \
  -d '{"email":"comprador@agrocolab.cl","password":"123","rol":"comprador"}'
# 0. Pagar el plan. Anotar pago.id
curl -i -X POST $API/pagos -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"tipo":"plan","plan_nombre":"Canasta Huerto Esencial","numero":"4242 4242 4242 4242",
       "titular":"Camila Soto","vencimiento":"12/30","cvv":"123"}'
# 1. CREAR el pedido con el pago.id anterior
curl -i -X POST $API/pedidos -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"plan_nombre":"Canasta Huerto Esencial","cliente_nombre":"Camila Soto",
       "telefono":"+56911111111","email":"comprador@agrocolab.cl","tipo_destino":"particular",
       "direccion":"Av. Falsa 123","comuna":"Ñuñoa","frecuencia_entrega":"mensual","pago_id":1}'
# 2. LEER: el dato quedó guardado
curl -i $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
# 3. ACTUALIZAR
curl -i -X PUT $API/pedidos/1 -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"tipo_destino":"particular","direccion":"Av. Falsa
123","comuna":"Macul","frecuencia_entrega":"mensual"}'
# 4. BORRAR y volver a leer: responde 404
curl -i -X DELETE $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
curl -i $API/pedidos/1 -H "Authorization: Bearer $TOKEN"
```

Anexo C. Script SQL equivalente del esquema

Es el archivo `db/esquema.sql`, generado desde los modelos de `backend/database.py` con el compilador de SQLAlchemy para PostgreSQL. La aplicación crea este esquema al arrancar.

```
CREATE TABLE eventos_seguridad (
  id SERIAL NOT NULL,
  momento TIMESTAMP WITHOUT TIME ZONE,
  tipo VARCHAR,
  severidad VARCHAR,
  ip VARCHAR,
  usuario_id INTEGER,
  email VARCHAR,
  ruta VARCHAR,
  detalle VARCHAR,
  PRIMARY KEY (id)
);
CREATE INDEX ix_eventos_seguridad_ip ON
eventos_seguridad (ip);
CREATE INDEX ix_eventos_seguridad_momento ON
eventos_seguridad (momento);
CREATE INDEX ix_eventos_seguridad_tipo ON
eventos_seguridad (tipo);

CREATE TABLE intentos_acceso (
  id SERIAL NOT NULL,
  clave VARCHAR,
  momento FLOAT,
  PRIMARY KEY (id)
);
CREATE INDEX ix_intentos_acceso_clave ON
intentos_acceso (clave);
CREATE INDEX ix_intentos_acceso_momento ON
intentos_acceso (momento);

CREATE TABLE usuarios (
  id SERIAL NOT NULL,
  nombre VARCHAR,
  email VARCHAR,
  password VARCHAR,
  rol VARCHAR,
  telefono VARCHAR,
  direccion VARCHAR,
  verificado INTEGER,
  codigo_verificacion VARCHAR,
  codigo_expira TIMESTAMP WITHOUT TIME ZONE,
  recuperacion_hash VARCHAR,
  recuperacion_expira TIMESTAMP WITHOUT TIME ZONE,
  recuperacion_intentos INTEGER,
  consentimiento_datos INTEGER,
  fecha_consentimiento TIMESTAMP WITHOUT TIME ZONE,
  acepta_terminos INTEGER,
  politica_privacidad TEXT,
  PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_usuarios_email ON usuarios
(email);
CREATE INDEX ix_usuarios_id ON usuarios (id);
CREATE INDEX ix_usuarios_nombre ON usuarios (nombre);

CREATE TABLE arboles (
  id SERIAL NOT NULL,
  nombre VARCHAR,
  ubicacion VARCHAR,
  plan VARCHAR,
  tipo_suscripcion VARCHAR,
  cupos_totales INTEGER,
  cupos_ocupados INTEGER,
  precio_mensual_clp INTEGER,
  estimacion_cosecha VARCHAR,
  imagen VARCHAR,
  especie VARCHAR,
  identificador VARCHAR,
```

```
  imagen_origen VARCHAR,
  productor_id INTEGER,
  PRIMARY KEY (id),
  FOREIGN KEY(productor_id) REFERENCES usuarios (id)
);
CREATE INDEX ix_arboles_id ON arboles (id);
CREATE INDEX ix_arboles_nombre ON arboles (nombre);

CREATE TABLE pagos (
  id SERIAL NOT NULL,
  usuario_id INTEGER,
  tipo VARCHAR,
  arbol_id INTEGER,
  plan_nombre VARCHAR,
  concepto VARCHAR,
  monto_clp INTEGER,
  estado VARCHAR,
  motivo_rechazo VARCHAR,
  marca VARCHAR,
  ultimos4 VARCHAR,
  codigo_autorizacion VARCHAR,
  transaccion VARCHAR,
  pedido_id INTEGER,
  fecha TIMESTAMP WITHOUT TIME ZONE,
  PRIMARY KEY (id),
  FOREIGN KEY(usuario_id) REFERENCES usuarios (id),
  UNIQUE (transaccion)
);
CREATE INDEX ix_pagos_id ON pagos (id);
CREATE INDEX ix_pagos_usuario_id ON pagos
(usuario_id);

CREATE TABLE pedidos (
  id SERIAL NOT NULL,
  cliente_id INTEGER,
  plan_nombre VARCHAR,
  monto_clp INTEGER,
  tipo_destino VARCHAR,
  razon_social VARCHAR,
  rut VARCHAR,
  direccion VARCHAR,
  comuna VARCHAR,
  frecuencia_entrega VARCHAR,
  codigo_seguimiento VARCHAR,
  fecha_registro TIMESTAMP WITHOUT TIME ZONE,
  estado_logistica VARCHAR,
  tipo VARCHAR,
  arbol_id INTEGER,
  PRIMARY KEY (id),
  FOREIGN KEY(cliente_id) REFERENCES usuarios (id),
  UNIQUE (codigo_seguimiento),
  FOREIGN KEY(arbol_id) REFERENCES arboles (id)
);
CREATE INDEX ix_pedidos_id ON pedidos (id);

CREATE TABLE suscripciones (
  id SERIAL NOT NULL,
  usuario_id INTEGER,
  arbol_id INTEGER,
  fecha_suscripcion TIMESTAMP WITHOUT TIME ZONE,
  PRIMARY KEY (id),
  CONSTRAINT uq_suscripcion_usuario_arbol UNIQUE
(usuario_id, arbol_id),
  FOREIGN KEY(usuario_id) REFERENCES usuarios (id),
  FOREIGN KEY(arbol_id) REFERENCES arboles (id)
);
CREATE INDEX ix_suscripciones_id ON suscripciones
(id);
```

Anexo D. Procedimiento de medición

Tabla 13. Entorno y procedimiento de las mediciones del 4 de noviembre de 2026.

Aspecto	Detalle
Código	El de esta entrega; dependencias desde `requirements-dev.txt` en un entorno virtual nuevo. La instalación se repitió en una copia limpia siguiendo el README.
Equipo	Intel Core i5-12450H, 12 hilos, 15 GB de RAM, Linux. Cliente, servidor y base en el mismo equipo.
Base de datos	PostgreSQL 16.2 temporal por socket Unix, y SQLite para la segunda ejecución de las pruebas.
Servidor	Uvicorn con 4 procesos para concurrencia y carga. Sin proxy ni TLS.
Concurrencia	Cliente asíncrono `httpx` con pagos lanzados a la vez. Tras cada ronda se consultaron `cupos_ocupados`, suscripciones y pagos aprobados.
Carga	300 peticiones por combinación de lectura; 150 y 140 pagos con compradores distintos. Clientes concurrentes: 1, 10 y 50 en lecturas; 1 y 10 en pagos.
Cobertura	`pytest --cov` sobre el back end, sin contar las pruebas: 1.408 de 1.551 sentencias.
Códigos HTTP	29 peticiones secuenciales con datos ficticios, antes y después de la corrección; se registraron el código y el campo `status`.

Aspecto	Detalle
Cliente	Navegador Chrome contra el servicio local: inicio de sesión por la interfaz, y pago, cancelación y eliminación de cuenta con el cliente de la aplicación, sin errores de JavaScript.